📄 Resume Management System – Full Stack Documentation

1. Project Overview

The Resume Management System is a full-stack application designed to manage resume files efficiently. It allows users to upload, store, view, download, and delete resumes.

The system is built using:
Backend: Spring Boot (Java 17)
Frontend: Angular 18
Database: MongoDB
Storage: Local File System
The architecture follows a clean separation of concerns with scalable design principles.

2. Architecture

🔹 High-Level Architecture

Client (Angular UI)
↓
REST API (Spring Boot)
↓
Service Layer (Business Logic)
↓
Repository Layer
↓
MongoDB (Metadata Storage)

Service Layer → Local File System (Resume Files)

🔹 Frontend Architecture (Angular)

Standalone Components
Feature-based folder structure

```
src/app/
│
├── core/services → API calls
├── features/resume
│   ├── upload → Upload UI
│   ├── list → Resume list UI
│
├── shared/models → Data models
```

3. Backend (Spring Boot)

🔹 Responsibilities

Handle file upload/download
Validate file type & size
Store files in local system
Store metadata in MongoDB
Provide REST APIs

🔹 Features Implemented

✔ Upload Resume (PDF/DOC/DOCX)
✔ Store file in local system
✔ Store metadata in MongoDB
✔ Fetch all resumes
✔ Download resume
✔ Delete resume (file + DB)

🔹 Database Design

Database: biz
Collection: resumes
Sample Document:

```
{
"_id": "123",
"fileName": "resume.pdf",
"fileType": "application/pdf",
```

"filePath": "/path/to/file",
"fileSize": 70965,
"uploadedAt": "2026-03-25T02:12:58"
}

🔹 File Storage
Stored in:
~/workspace/docs/resume/

Unique naming using UUID to prevent overwrite

4. Frontend (Angular)
🔹 Responsibilities
Upload file via UI
Show resume list
Preview (PDF only)
Download & delete resumes
Provide validation & feedback

🔹 Features Implemented
✔ Upload UI with validation
✔ File type & size validation
✔ Toast notifications (auto-hide)
✔ Loading spinner during upload
✔ Resume listing table
✔ Empty state UI
✔ File type badges (PDF/DOC/DOCX)
✔ Preview (PDF only)
✔ Download functionality
✔ Delete with confirmation
✔ Disabled preview for non-PDF

🔹 UI/UX Enhancements

Bootstrap-based responsive design
Clean navbar and layout
Disabled upload button for invalid files
Error handling with user-friendly messages

5. API Contracts

🔹 Upload Resume

POST /api/resumes/upload

Request: form-data (file)
Response: Resume metadata

🔹 Get All Resumes

GET /api/resumes

🔹 Download Resume

GET /api/resumes/download/{id}

Response: File (Blob)

🔹 Delete Resume

DELETE /api/resumes/{id}

6. Data Flow

🔹 Upload Flow

User selects file in UI
Frontend validates file
API call to backend
Backend validates again
File saved to system
Metadata stored in MongoDB
Response returned

- Download / Preview Flow

User clicks preview/download
Request sent to backend
Backend fetches metadata
File read from system
Returned as Blob
Opened in browser (preview) or downloaded

- Delete Flow

User clicks delete
Confirmation shown
API call sent
File deleted from system
Record removed from MongoDB
UI refreshed

7. Setup Instructions

- Backend Setup

Clone project
Configure MongoDB:
spring.data.mongodb.uri=mongodb://localhost:27017/biz

Set file path:
file.upload.dir=~/workspace/docs/resume/

Run:
mvn spring-boot:run

- Frontend Setup

Install dependencies:
npm install

Run Angular:
ng serve -o

8. Future Enhancements

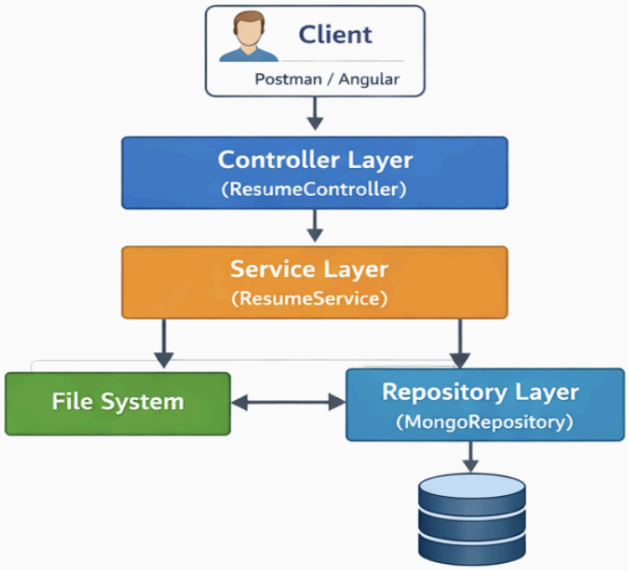
- ☐ JWT Authentication & Authorization
- ☐ Role-based access control
- ☐ AWS S3 integration (cloud storage)
- ☐ Resume search & filtering
- ☐ Pagination
- ☐ Drag & drop upload
- ☐ AI-based resume parsing

9. Conclusion

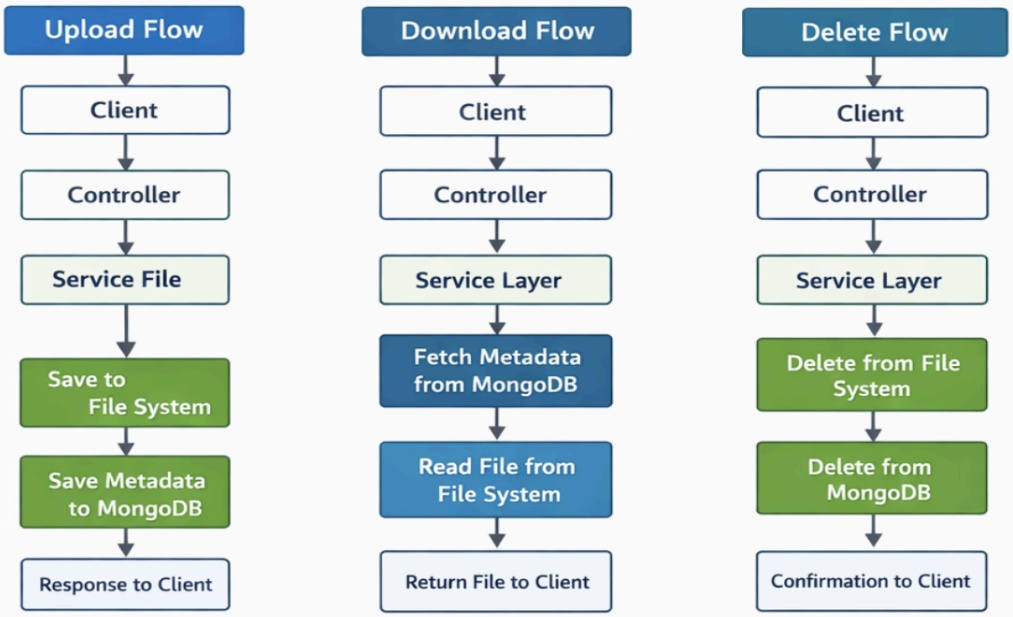
The Resume Management System is a fully functional full-stack application demonstrating clean architecture, proper validation, and modern UI/UX practices.

It handles the complete lifecycle of resume files and is scalable for real-world production enhancements.

Architecture



Data Flows



📄 Backend Dockerization Documentation

1. Overview
The backend of the Resume Project is developed using Spring Boot and packaged as an executable JAR file. To ensure portability and consistency across environments, the backend is containerized using Docker.

2. Prerequisites
Before Dockerizing the backend, the following requirements must be met:

- Java installed (JDK 17)
- Docker Desktop installed and running
- Maven Wrapper (mvnw) available in the project
- A valid pom.xml file

3. Building the JAR File
The Spring Boot application is first converted into an executable JAR file using Maven.

Command: ./mvnw clean package

Explanation: clean: Removes previous build artifacts, package: Compiles code and generates the JAR file

Output: The generated JAR file is located inside the target/ directory:
target/resume-0.0.1-SNAPSHOT.jar

Verification: The JAR file is tested locally before Dockerization:
java -jar target/resume-0.0.1-SNAPSHOT.jar

4. Dockerfile Creation
A Dockerfile is created to define how the backend application will run inside a container.

Dockerfile:
```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5. Dockerfile Explanation

- FROM eclipse-temurin:17-jdk

Uses an official OpenJDK 17 base image maintained by Eclipse Temurin.

- WORKDIR /app

Sets the working directory inside the container.

- COPY target/*.jar app.jar

Copies the built JAR file into the container.

- EXPOSE 8080

Indicates that the application runs on port 8080.

- ENTRYPOINT ["java", "-jar", "app.jar"]

Defines the command to run the application when the container starts.

6. Building the Docker Image
The Docker image is created using the Dockerfile.

Command: docker build -t resume-backend .

Explanation:
docker build: Builds the image
-t resume-backend: Tags the image with a name
. : Uses current directory as build context

7. Running the Docker Container
The Docker image is executed as a container.
Command: docker run -p 8080:8080 resume-backend

Explanation:
docker run: Starts a container
-p 8080:8080: Maps container port to host port
resume-backend: Image name

8. Application Access
Once the container is running, the backend application can be accessed at:
spring.data.mongodb.uri=mongodb://mongo:27017/biz

9. Key Benefits of Dockerization

- Ensures consistent environment across systems
- Eliminates dependency issues
- Simplifies deployment process
- Enables scalability and portability

10. Conclusion
The backend application is successfully containerized using Docker. The application can now run consistently across different environments using a single Docker command without requiring manual setup

📄 FRONTEND DOCKERIZATION DOCUMENTATION

1. Overview
The frontend of the Resume Project is built using Angular. The production build generates static files, which are served using Nginx inside a Docker container.

2. Prerequisites

- Node.js installed
- Angular project setup
- Docker installed and running

3. Building the Angular Application

Command:
npm install
npm run build

👉 Output is generated inside: dist/resume-frontend/browser/

4. Dockerfile Creation

```
FROM nginx:alpine
COPY dist/resume-frontend/browser /usr/share/nginx/html
RUN mv /usr/share/nginx/html/index.csr.html /usr/share/nginx/html/index.html
EXPOSE 80
```

5. Dockerfile Explanation

FROM nginx:alpine
Lightweight web server to serve static files
COPY dist/...
Copies Angular build files into Nginx directory
RUN mv index.csr.html
Renames SSR file to standard index.html
EXPOSE 80
Nginx runs on port 80

6. Building Docker Image

```
docker build -t resume-frontend .
```

7. Running Container
docker run -p 4200:80 resume-frontend

8. Application Access
http://localhost:4200

9. Key Benefits
- No Node.js needed in production
- Lightweight and fast
- Optimized static delivery

10. Conclusion
The Angular frontend is containerized and served efficiently using Nginx, ensuring high performance and portability.

📄 DATABASE (MONGODB) DOCUMENTATION

1. Overview
MongoDB is used as the database for storing resume data. It is run as a Docker container for easy setup and isolation.

2. Running MongoDB Container
docker run -d -p 27017:27017 --name mongo mongo

3. Explanation

-d → run in background
-p 27017:27017 → port mapping
mongo → official image

4. Access
Host: localhost
Port: 27017
Database: biz

5. Docker Networking (IMPORTANT)
Inside Docker:
Backend connects using → mongo:27017

Outside Docker:
Local app connects using → localhost:27017

6. Benefits
- No manual Mongo installation
- Easy setup and teardown
- Isolated environment

7. Conclusion
MongoDB runs as a containerized service and integrates seamlessly with backend using Docker networking.

📄 DOCKER COMPOSE DOCUMENTATION (MOST IMPORTANT)

1. Overview
Docker Compose is used to manage multi-container applications including:

1. Frontend
2. Backend
3. MongoDB

2. docker-compose.yml

```
version: '3.8'

services:

mongo:
image: mongo
container_name: mongo
ports:
- "27017:27017"
volumes:
- mongo-data:/data/db

backend:
build: ./backend/resume
container_name: backend
ports:
- "8080:8080"
depends_on:
- mongo
environment:
- SPRING_PROFILES_ACTIVE=docker
volumes:
- ~/workspace/docs/resume:/app/uploads

frontend:
build: ./frontend/resume-frontend
container_name: frontend
ports:
```

```
- "4200:80"
depends_on:
- backend

volumes:
mongo-data:
```

3. Running Entire Application
docker-compose up --build

4. Stopping Application
docker-compose down

5. Advantages

- Single command startup
- Automatic networking
- Dependency management
- Clean environment setup

6. Architecture Flow
Browser → Frontend (Nginx)
→ Backend (Spring Boot)
→ MongoDB

7. Conclusion
Docker Compose simplifies deployment by orchestrating all services together, ensuring seamless communication and easy management.